**Version: 1.0**



# Selection

## C-Pointers


**Résumé**

Entraînez-vous aux opérations fondamentales sur les pointeurs en C, y compris l'affectation de valeurs, l'échange, les opérations arithmétiques et la manipulation de tableaux.


#C

#Pointers

#Memory-Management



# Chapitre 3

# Tutoriel

- Il est maintenant temps de parler de la partie magique de la programmation en C : **LES POINTEURS** ! Un **pointeur** est un type spécial en `C` qui contient une adresse. Chaque octet en RAM a sa propre adresse individuelle, et un pointeur stocke l'adresse d'un autre octet en RAM.
- Un `pointeur` n'est pas autonome - vous avez toujours un pointeur vers un autre type `C`. Exemple : vous pouvez avoir un pointeur sur un caractère, ou un pointeur sur un entier. Un pointeur sur un entier stocke l'adresse d'un entier existant ailleurs en mémoire.
- Vous devriez déclarer un pointeur comme suit : `<type> *<nom_var>`. Par exemple : `int *pointeur_sur_entier;`



Un pointeur ne contient pas une valeur comme les autres types. Il contient une adresse où une valeur est stockée. Si vous ne comprenez pas pleinement, vous devez vérifier votre compréhension avec l'un de vos voisins.

- Créez un répertoire de travail et écrivez un fichier `C` simple. Déclarez un pointeur sur un entier et compilez-le pour vérifier la syntaxe :

**main.c**

```
int main()
{
    int *pointeur_sur_entier;
    return(0);
}
```

- Vous pouvez assigner une adresse au pointeur en utilisant l'opérateur `&` pour obtenir l'adresse d'une variable existante :

© 2026 Association 42. - Tous droits réservés

**Exemple**

```
int a;
int *p;
p = &a;
```

- Ajoutez une variable `int`, un pointeur `int`, et assignez l'adresse de l'`int` au pointeur. Compilez et testez.
- Vous pouvez utiliser le pointeur pour lire et mettre à jour l'autre variable. Cela s'appelle le "déréférencement" en utilisant l'opérateur `*`. Par exemple, `*p` peut être utilisé comme une variable `int` régulière.
- Assignez une valeur à votre variable `int` et imprimez-la en utilisant votre fonction `ft_putnbr`.
- Maintenant, imprimez la même valeur mais en utilisant le pointeur au lieu du nom de la variable : `ft_putnbr(*p);`. Vérifiez que les deux impressions montrent la même valeur.
- Mettez à jour l'`int` en utilisant le nom de la variable, puis imprimez à nouveau en utilisant à la fois la variable et le pointeur. Qu'observez-vous ?
- Mettez à jour l'`int` en utilisant le pointeur : `*p = 100;` Puis imprimez à nouveau en utilisant les deux méthodes. Que se passe-t-il ?



Rendons cela simple pour `int *p;` :
`*p` est une variable de type `int` dont l'adresse est stockée dans `p`.

- Maintenant, explorons les `chaînes de caractères`. En `C`, une chaîne de caractères est stockée comme une série d'octets en RAM, chacun contenant la valeur `ASCII` de chaque caractère. La chaîne se termine par un octet contenant 0.
- Vous pouvez créer une chaîne de caractères en utilisant des guillemets doubles : `"Bonjour"`. Cela trouve un espace mémoire et stocke les codes ASCII. L'expression évalue à l'adresse du premier caractère.
- Déclarez un pointeur `char` et assignez-lui une `chaîne de caractères` :

**Exemple**

```
char *s;

s = "Bonjour";
```

- Utilisez `*s` pour accéder au premier `char` et imprimez-le avec `ft_putchar(*s);`.
- Voici l'« arithmétique des pointeurs » ! Si `s` pointe vers le premier `char`, alors `s+1` pointe vers le deuxième `char`. Essayez ce qui suit :

© 2026 Association 42. - Tous droits réservés

**Exemple**

```
char *s;

s = "Bonjour";
s = s + 1;
ft_putchar(*s);
```

- Maintenant, essayez :

**Exemple**

```
char *s;

s = "Bonjour";
ft_putchar(*(s+1));
```

Vérifiez avec l'un de vos pairs si vous voyez une différence.

- L'arithmétique des pointeurs dépend du type pointé. `char_ptr+1` pointe vers le `char` suivant, `int_ptr+1` pointe vers l'`int` suivant (4 octets plus loin).
- Expérimentez en parcourant une chaîne de caractères caractère par caractère en utilisant l'arithmétique des pointeurs. Imprimez chaque caractère individuellement.
- Essayez ce mystère : écrivez une fonction qui reçoit un pointeur `int` en paramètre et modifie la valeur vers laquelle il pointe. Appelez cette fonction depuis le main et vérifiez que la variable originale a été modifiée.

© 2026 Association 42. - Tous droits réservés

# Chapitre 4

# Exercice 0 : ft_ft

|  | Exercice: 0 | |
|---|---|---|
| ft_ft | | |
| Répertoire: *ex*0/ | | |
| Fichiers à Rendre: `ft_ft.c` | | |
| Autorisé: `None` | | |

- Créez une fonction qui prend un pointeur sur un `int` en paramètre, et affecte la valeur "42" à ce `int`.

**Prototype :**

```
void ft_ft(int *nbr);
```

© 2026 Association 42. - Tous droits réservés

# Chapitre 5

# Exercice 1 : ft_ultimate_ft

|  | Exercice: 1 | |
|---|---|---|
| ft_ultimate_ft | | |
| Répertoire: *ex1/* | | |
| Fichiers à Rendre: `ft_ultimate_ft.c` | | |
| Autorisé: `None` | | |

- Créez une fonction qui prend un pointeur de pointeur de pointeur de pointeur de pointeur de pointeur de pointeur de pointeur de pointeur de pointeur sur un `int` en paramètre, et affecte la valeur "42" à ce `int`.

**Prototype :**

```
void ft_ultimate_ft(int *********nbr);
```

© 2026 Association 42. - Tous droits réservés

# Chapitre 6

# Exercice 2 : ft_swap

|  | Exercice: 2 | |
|---|---|---|
| ft_swap | | |
| Répertoire: *ex2/* | | |
| Fichiers à Rendre: `ft_swap.c` | | |
| Autorisé: `None` | | |

- Créez une fonction qui échange la valeur de deux entiers dont les adresses sont passées en paramètre.

**Prototype :**

```
void ft_swap(int *a, int *b);
```

© 2026 Association 42. - Tous droits réservés

# Chapitre 7

# Exercice 3 : ft_div_mod

|  | Exercice: 3 | |
|---|---|---|
| ft_div_mod | | |
| Répertoire: *ex*3/ | | |
| Fichiers à Rendre: `ft_div_mod.c` | | |
| Autorisé: `None` | | |

- Créez une fonction `ft_div_mod`.

**Prototype :**

```
void ft_div_mod(int a, int b, int *div, int *mod);
```

- Cette fonction divise le paramètre `a` par `b` et stocke le résultat dans l'entier pointé par `div`. Elle stocke également le reste de la division de `a` par `b` dans l'entier pointé par `mod`.

© 2026 Association 42. - Tous droits réservés

# Chapitre 8

# Exercice 4 : ft_ultimate_div_mod

|  | Exercice: 4 | |
|---|---|---|
| ft_ultimate_div_mod | | |
| Répertoire: *ex*4/ | | |
| Fichiers à Rendre: `ft_ultimate_div_mod.c` | | |
| Autorisé: `None` | | |

- Créez une fonction `ft_ultimate_div_mod`.

**Prototype :**

```
void ft_ultimate_div_mod(int *a, int *b);
```

- Cette fonction divise le contenu pointé par `a` par celui pointé par `b`.
  - Le résultat est stocké dans l'entier pointé par `a`.
  - Le reste de la division est stocké dans l'entier pointé par `b`.

© 2026 Association 42. - Tous droits réservés

# Chapitre 9

# Exercice 5 : ft_rev_int_tab

|  | Exercice: 5 | |
|---|---|---|
| ft_rev_int_tab | | |
| Répertoire: *ex*5/ | | |
| Fichiers à Rendre: `ft_rev_int_tab.c` | | |
| Autorisé: `None` | | |

- Créez une fonction qui inverse un tableau d'entiers donné (le premier devient le dernier, etc).

**Prototype :**

```
void ft_rev_int_tab(int *tab, int size);
```

- Les arguments sont un pointeur sur un `int` et le nombre d'éléments dans le tableau.

© 2026 Association 42. - Tous droits réservés

# Chapitre 10

# Exercice 6 : ft_sort_int_tab

|  | Exercice: 6 | |
|---|---|---|
| ft_sort_int_tab | | |
| Répertoire: *ex6/* | | |
| Fichiers à Rendre: `ft_sort_int_tab.c` | | |
| Autorisé: `None` | | |

- Créez une fonction qui trie un tableau d'entiers par ordre croissant.

**Prototype :**

```
void ft_sort_int_tab(int *tab, int size);
```

- Les arguments sont un pointeur sur un `int` et le nombre d'éléments dans le tableau.

© 2026 Association 42. - Tous droits réservés